

Object Oriented Scraping (CodeGrade)

 [_ \(https://github.com/learn-co-curriculum/python-p4-scraping-code-along\)](https://github.com/learn-co-curriculum/python-p4-scraping-code-along)  [_ \(https://github.com/learn-co-curriculum/python-p4-scraping-code-along/issues/new\)](https://github.com/learn-co-curriculum/python-p4-scraping-code-along/issues/new)

Learning Goals

- Scrape a web page's HTML using BeautifulSoup.
 - Use scraped data to give attributes to Python objects.
 - Write an object oriented Scraper class.
-

Key Vocab

- **Web scraping:** Web scraping, web harvesting, or web data extraction is data scraping used for extracting data from websites
 - **HTML:** The HyperText Markup Language or HTML is the standard markup language for documents designed to be displayed in a web browser.
 - **CSS:** Cascading Style Sheets is a style sheet language used for describing the presentation of a document written in a markup language such as HTML or XML.
 - **CSS selector:** CSS selectors define the elements to which a set of CSS rules apply.
-

Introduction

One of the most common use-cases for web scraping involves you, the programmer, scraping data that you will then use to instantiate your own Python objects. In this lab, we'll be scraping a static site that displays the course offerings of the Flatiron School. We'll be taking the

scraped data to create our own `Course` objects. Each instance of the `Course` class will have a series of attributes. The values of each of these attributes will be scraped from the website.

We'll be creating two classes, `Scraper` and `Course`. The `Scraper` class will be responsible for using BeautifulSoup to scrape the relevant data. It will also be responsible for taking that data and using it to instantiate instances of the `Course` class.

Code Along I: The `Course` Class

Fork and clone this lab to get started!

Before we build our `Scraper`, we'll build the `Course` class. We know that the purpose of our scrape is to get data to assign to the attributes of `Course` class instances.

Let's take a look at the site we'll be scraping in order to get a sense of what attributes we should give our `Course` class instances. Go ahead and open up <http://learn-co-curriculum.github.io/site-for-scraping/courses>  (<http://learn-co-curriculum.github.io/site-for-scraping/courses>).

Scroll down until you are looking at the list of course offerings:

course list

We can see that each course has a title, a schedule (either Part- or Full-Time) and a description. This seems like a great place to start in terms of defining our own `Course` class objects.

Open up `lib/Course.py` and define your class:

```
class Course:
    pass
```

Now, let's run *just* the `Course` specs by typing `pytest testing/course_test.py` in the terminal. You should see the following test output:

```
=====
self = <testing.course_test.Test_Course object at 0x1038a79d0>
```

```
def test_title_instance_variable(self):
    "Test setting and getting for title"
>     course = Course(title="Programming Robots for Outer Space")
E     TypeError: Course() takes no arguments
```

```
testing/course_test.py:10: TypeError
```

```
-----
```

```
self = <testing.course_test.Test_Course object at 0x1038a7b80>
```

```
def test_schedule_instance_variable(self):
    "Test setting and getting for schedule"
>     course = Course(schedule="Full-Time")
E     TypeError: Course() takes no arguments
```

```
testing/course_test.py:15: TypeError
```

```
-----
```

```
self = <testing.course_test.Test_Course object at 0x1038a78e0>
```

```
def test_description_instance_variable(self):
    "Test setting and getting for description"
>     course = Course(description="Learn how to program robots to explore the depths of space. Guest lecturer: The
E     TypeError: Course() takes no arguments
```

```
testing/course_test.py:20: TypeError
```

```
=====
```

FAILED Course in Course.py Test setting and getting for title - TypeError: Course() takes no arguments

FAILED Course in Course.py Test setting and getting for schedule - TypeError: Course() takes no arguments

Looks like we have some methods to define. Let's start with the instance variable tests. The test output tells us that we need setters and getters for `title`, `schedule` and `description`. Lets also add the `__str__` function to represents the Course objects as a string.

```
class Course:
    def __init__(self, title, schedule, description):
        self.title = title
        self.schedule = schedule
        self.description = description

    def __str__(self):
        output = ''
        output += f'Title: {self.title}\nSchedule: {self.schedule}\nDescription: {self.description}\n'
        output += '-----'
        return output
```

Go ahead and run the test suite again. Now we should be passing all of our instance method tests!

Code Along II: The `Scraper` Class

Start by running the `Scraper` specs with the `pytest testing/scraper_test.py` line in your terminal. You should see failing tests and describe a number of methods. Let's run through the desired behavior of each method:

`get_page()`

The `get_page()` instance method will be responsible for using Beautiful Soup and `requests` to grab the entire HTML document from the web page.

get_courses()

The `get_courses()` instance method will be responsible for using a CSS selector to grab all of the HTML elements that contain a course. In other words, the return value of this method should be a collection of BeautifulSoup objects, each of which describes a course offering. We're going to have to examine the page with the element inspector to find the CSS selector that contains the courses.

make_courses()

The `make_courses()` method will be responsible for actually instantiating `Course` objects and giving each course object the correct `title`, `schedule` and `description` attribute that we scraped from the page.

print_courses()

The `print_courses()` method we made for you! It calls on `.make_courses` and then iterates over all of the courses that get created to `print` out a list of course offerings using the `__str__` method we wrote for the `Course` class. We gave you this freebie so that we can easily see how cool it is to scrape data and make real live Python objects with it.

Now that we have a basic concept of the methods we're expected to build, we're going to ignore them (surprise!). We've already discussed how tricky it is to scrape data from a web page. It is a very precise process and it takes *a lot* of playing around in `ipdb` to find the right CSS selectors for the desired data. So, we're going to start by building our `get_page()` method. **As soon as we get the HTML document using BeautifulSoup, we will drop into our program using `ipdb` and play around with CSS selectors until we find what we're looking for.** Once we have working code, we'll worry about organizing the appropriate code into the above-described methods.

Getting the HTML Doc and Finding Our Selectors

Open up `lib/Scraper.py` and define the `.get_page` method:

```
from bs4 import BeautifulSoup
import requests
import ipdb
```

```
class Scraper:

    def __init__(self):
        self.courses = []

    def get_page(self):
        # more code coming soon!
```

Notice that we are already requiring BeautifulSoup, requests and ipdb at the top of the file. We are ready to use BeautifulSoup and requests to get our HTML. Add the following line to your `.get_page` method:

```
doc = BeautifulSoup(
    requests.get(
        "http://learn-co-curriculum.github.io/site-for-scraping/courses"
    ).text,
    'html.parser'
)
```

Then, we'll place a `ipdb.set_trace()` on the next line. At the bottom of the file, outside of the class definition, we'll call `Scraper.get_page()`. That way, we'll hit our binding and be able to play around with the HTML document in the terminal in order to find the CSS selectors we're looking for:

```
from bs4 import BeautifulSoup
import requests
import ipdb
import Course
import json
```

```
class Scraper:
```

```
def __init__(self):
    self.courses = []
def get_page():
    # more code coming soon!
    doc = BeautifulSoup(requests.get("http://learn-co-curriculum.github.io/site-for-scraping/courses").text, 'r
    ipdb.set_trace()
```

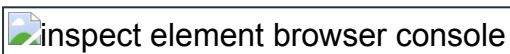
```
Scraper.get_page()
```

Once your file looks like the code above, run the file with `python lib/Scraper.py` in your terminal, type the `doc.content` variable into the terminal and you should see the HTML document, retrieved for us by BeautifulSoup and requests. You should see something like this:

```
b'<!DOCTYPE html> <html> <head> <meta charset=utf-8> <meta name=viewport content="width=device-width, initial-scale=
...'
```

Okay, we're ready to find the CSS selector that will grab the course offering from the HTML document. How should we go about doing this? Should we guess? Should we manually read the entire HTML document, looking for the HTML elements that contain the course offerings? Nope. We're going to revisit the Flatiron website in the browser and use the developer tools of our browser to inspect the elements.

Click on [this link](http://learn-co-curriculum.github.io/site-for-scraping/courses) and once again scroll down to the section of the page that lists the course offerings. Right click on any course offering and select "inspect element". You should see something like this in your browser:



Let's take a closer look at the highlighted line in the element inspector:

```
<article class="post same-height-left" style="height: 489px;">
```

Looks like the element that contains an individual course has a class of "post". Let's use this CSS selector of `.post` to try to grab *all* courses.

Go back to your terminal and execute the following line:

```
doc.select(".post")
```

You should see something like this:

```
ipdb> doc.select(".post")
[<article class="post">  <h2>Web
```

Whoa! That's a lot of results. But, if you take a closer look at the content, you'll see that this list of tag (`bs4.element.Tag`) elements do describe the individual courses. You'll notice course titles and descriptions, among other pieces of information.

Okay, now that we have a working line of code for grabbing all of the courses from the page, let's operate on those courses in order to find the title, schedule and description of each one.

Finding CSS Selectors for The Desired Attributes

We know that a collection of BeautifulSoup results element functions like a list. So, it makes sense that we can iterate over the collection in order to grab the title, schedule and description of each one. BUT, before we worry about iterating, let's grab *just one element* and try to identify the correct CSS selectors for title, schedule and description.

In your terminal, execute `doc.select(".post")[0]`. This will grab us just the first element from the collection. You should see something like this:

```
ipdb> doc.select(".post")[0]
<article class="post">  <h2>Web
```

This describes *just one course offering*. If you look closely, you'll see it contains all the info we need. You can see the title, the schedule and the description. The easiest way to ID the correct CSS selector for extracting this information, however, is to revisit the web page and examine a course offering with our "inspect element" tool.

Scraping Course Title

Go back to the [site](http://learn-co-curriculum.github.io/site-for-scraping/courses)  (<http://learn-co-curriculum.github.io/site-for-scraping/courses>) and open up the element inspector again. Click the symbol in the upper left of your console (it looks like an arrow cursor pointing into a box) to hover over the title of the first course offering. You should see a tag appear when you hover over the course title with this tool. The tag should say `h2 750.428 x 28px`.

We don't care about the height and width but we *do* care about the selector, `h2`.

Test the following code in your terminal:

```
doc.select(".post")[0].select("h2")
```

You should see the following returned to you:

```
[<h2>Web Development Immersive</h2>]
```

We're so close! The course title is right there, inside the Beautiful Soup tag element. Let's grab it:

```
doc.select(".post")[0].select("h2")[0].text
```

You should see the following return value:

```
"Web Development Immersive"
```

We did it! We found the code for grabbing an individual course's title. Let's do the same for schedule and description.

Scraping Course Schedule

Go back to the [site](http://learn-co-curriculum.github.io/site-for-scraping/courses)  (<http://learn-co-curriculum.github.io/site-for-scraping/courses>) and open up the element inspector again. Use the magnifying glass symbol to hover over the schedule of the first course offering. You should see a tag appear when you hover over the schedule (the line that reads "Part-Time" or "Full-Time") that reads `em.date ...`. It looks like the schedule element has a class of "date". Let's use that CSS selector to grab the date of the first course. In your terminal, execute:

```
doc.select('.post')[0].select('.date')[0].text
```

You should see the following returned to you:

```
'Full-Time'
```

Great, now we have the code for grabbing an individual course's schedule. Let's get that description.

Scraping Course Description

Once again, use the magnifying glass to hover over the first course's description. You should see a tag appear with the following text: `p 750`. `blah blah some pixels`. Okay, it looks like we have our selector: the `p` tag. Try out the following line in your console:

```
doc.select('.post')[0].select('p')[0].text
```

You should see returned to you:

```
"An intensive, Ruby and Javascript course that teaches the skills necessary to start a career as a full-stack software engineer"
```

We did it! We have the working code for grabbing:

- The page itself:
 - `doc = BeautifulSoup(requests.get("http://learn-co-curriculum.github.io/site-for-scraping/courses").text, 'html.parser')`
- The collection of course offerings:
 - `doc.select('.post')`
- The title of an individual course offering:
 - `doc.select(".post")[0].select("h2")[0].text`
- The schedule of an individual course offering:
 - `doc.select(".post")[0].select(".date")[0].text`

- The description of an individual course offering:
 - `doc.select(".post")[0].select("p")[0].text`

Now we're ready to use our code to create `Course` objects and give them attributes.

Creating `Course` Objects with Scraped Attributes

Notice that the `Scraper.py` file includes this line near the top:

```
from Course import Course
```

We are importing our `Course` class file so that our `Scraper` can make new courses and give them attributes scraped from the web page. We know how to grab an array-like collection of course elements from the page with the `doc.select(".post")` line. We also know what code will grab us the title, schedule and description of an individual member of that collection. So, we can iterate over the collection, make a new `Course` instance for each course offering element we are iterating over, and assign that instance the scraped title, schedule and description, using the working code for those attributes that we already figured out. In your `get_page()` method of the `Scraper` class, place the following code:

```
from bs4 import BeautifulSoup
import requests
from Course import Course
import json

class Scraper:
    def __init__(self):
        self.courses = []

    def get_page(self):
        doc = BeautifulSoup(requests.get("http://learn-co-curriculum.github.io/site-for-scraping/courses").text, 'r

        for course in doc.select('.post'):
            print(type(course))
```

```

title = course.select("h2")[0].text if course.select("h2") else ''
date = course.select(".date")[0].text if course.select(".date") else ''
description = course.select("p")[0].text if course.select("p") else ''

new_course = Course(title, date, description)
self.courses.append(new_course)

```

For each iteration over the collection of BeautifulSoup XML elements returned to us by the `doc.select(".post")` line, we are making a new instance of the `Course` class and giving that instance the `title`, `schedule` and `description` extracted from the tag object.

Place a `ipdb.set_trace()` at the end of the method. Now, run the code in this file with `python lib/Scraper.py`. When you hit the binding, enter `print(scraper.courses)` into your terminal and take a look at all the courses we made:

```

prabhdipgill@PrabhdiGillsMBP python-scraping-code-along % pipenv run python lib/Scraper.py
Title: Web Development Immersive
Schedule: Full-Time
Description: An intensive, Ruby and Javascript course that teaches the skills necessary to start a career as a full
-----
Title: iOS Development Immersive
Schedule: Full-Time
Description: An in-depth course on the iOS ecosystem that begins with Objective-C and Swift, explores popular iOS f
-----
Title: NYC Web Development Fellowship
Schedule: Full-Time
Description: In partnership with New York City, this free program trains NYC residents without college degrees for
-----
Title: Introduction to Front-End Development
Schedule: Part-Time
Description: An introductory course designed to teach students to launch fully responsive websites from scratch usi
-----
Title: Android for Developers

```

Schedule: Part-Time

Description: An advanced course that trains people with at least one year of software development experience in the

Title: Introduction to Data Science

Schedule: Part-Time

Description: A part-time course that provides students with the skills they need to make data meaningful.

Title: DIY Electronics - Build Your Own Sound Generating Circuit

Schedule: WORKSHOP

Description: This Beginners workshop teaches students the fundamentals of electricity, reading circuit diagrams, sol

Title:

Schedule:

Description:

Title:

Schedule:

Description:

Wow! We have a collection of `Course` objects, each of which have attributes that we scraped from the website. We are such good programmers.

Note that the last two courses are empty because the webpage has two empty placeholder courses with the `.post` css selector.

Extracting Our Code into Methods

Okay, we have some great working code. But, it doesn't really *all* belong in the `get_page()` method. The `get_page()` method should be responsible for *just getting the page*. Let's do some refactoring and get our `Scraper` tests passing!

`get_page()`

This method should contain *only the code for getting the HTML document*. Place the following code in your `get_page()` method and *comment out the rest of that method*. We'll need to refer to that code to get our other tests passing.

```
class Scraper:
    def __init__(self):
        self.courses = []

    def get_page(self):
        # more code coming soon!
        doc = BeautifulSoup(requests.get(
            "http://learn-co-curriculum.github.io/site-for-scraping/courses").text, 'html.parser')
        # for course in self.get_courses():

        #     title = course.select("h2")[0].text if course.select("h2") else ''
        #     date = course.select(".date")[0].text if course.select(".date") else ''
        #     description = course.select("p")[0].text if course.select("p") else ''

        #     new_course = Course(title, date, description)
        #     self.courses.append(new_course)
```

Run your `Scraper` test suite with `pytest testing/scrapper_test.py`. Your first test should be passing.

`get_courses()`

The `get_courses()` method should operate on the HTML page (which is the return value of the `.get_page` method) and return the collection of BeautifulSoup tag elements that describe each course. So, we'll call on our `.get_page` method inside the `.get_courses` method.

```
def get_courses(self):
    return self.get_page().select('.post')
```

Run the test suite again and the second test should be passing.

make_courses()

The `make_courses()` method should operate on the collection of course offering BeautifulSoup results list that was returned by the `.get_courses` method. The `.make_courses` method should iterate over the collection and make a new instance of `Course` class for each one while assigning it the appropriate attributes:

```
def make_courses(self):
    for course in self.get_page().select('.post'):

        title = course.select("h2")[0].text if course.select("h2") else ''
        date = course.select(".date")[0].text if course.select(".date") else ''
        description = course.select("p")[0].text if course.select("p") else ''

        new_course = Course(title, date, description)
        self.courses.append(new_course)
    return self.courses
```

Run the test suite again and all of your tests should be passing! Now, just for fun. Place the following line at the bottom of `lib/scrapper.py`

```
Scraper.print_courses()
```

Ta-da! We did it. Check out all of those awesome courses printed out to your terminal. If you're still having trouble getting your tests to pass, check out the final code below:

Solution Code: The `Scraper` Class

```
class Scraper:
    def __init__(self):
        self.courses = []

    def get_page(self):
        # more code coming soon!
```

```
doc = BeautifulSoup(requests.get(
    "http://learn-co-curriculum.github.io/site-for-scraping/courses").text, 'html.parser')
# ipdb.set_trace()
return doc

def get_courses(self):
    return self.get_page().select('.post')

def make_courses(self):
    for course in self.get_courses():

        title = course.select("h2")[0].text if course.select("h2") else ''
        date = course.select(".date")[0].text if course.select(".date") else ''
        description = course.select("p")[0].text if course.select("p") else ''

        new_course = Course(title, date, description)
        self.courses.append(new_course)
    return self.courses

def print_courses(self):
    for course in self.make_courses():
        print(course)
```

This tool needs to be loaded in a new browser window

Load Object Oriented Scraping (CodeGrade) in a new window